

Claude Code Practical Exercise

Use Claude Code (Antigravity) to complete each task below. Type the prompts exactly as shown, then tick the box when done. The goal is to feel what each term from the glossary actually does — not just read about it.

Before you start

Open a Claude Code session. All prompts go directly into Claude Code — not into Claude.ai chat.

Exercise 1 — .gitkeep

You will create an empty folder and discover why Git cannot see it — then fix it with a .gitkeep file.

Step 1 — Create an empty folder

Ask Claude Code to create a folder called my-archive with nothing inside it.

YOUR PROMPT

Create a folder called my-archive. Do not put any files inside it.

☐ Done — the folder exists on my computer

Step 2 — Ask Git what it sees

Initialise a Git repo and ask Claude Code what Git can see.

YOUR PROMPT

Initialise a Git repo in this folder. Then show me what Git sees with git status.

Pause and reflect

Look at the output. Does Git mention my-archive at all? Git cannot see an empty folder — it is invisible. This is the exact problem .gitkeep solves.

Step 3 — Fix it with .gitkeep

Add a .gitkeep file to make the folder visible to Git.

YOUR PROMPT

Add a .gitkeep file inside the my-archive folder. Then show me git status again.

☐ Git can now see the my-archive folder

- ☐ I understand why .gitkeep exists
-

Exercise 2 — .gitignore

You will create a .gitignore file and prove that it hides files from Git completely.

Step 1 — Create a secret file

Create a file that should never be shared — a fake .env with made-up credentials.

YOUR PROMPT

Create a file called .env with this
content:ANTHROPIC_API_KEY=fake-secret-key-123NOTION_API_KEY=fake-notion-456

- ☐ The .env file exists

Step 2 — Check Git can see it (danger!)

Before we protect it, confirm Git is currently tracking this secret file.

YOUR PROMPT

Show me git status. Can Git see the .env file right now?

Pause and reflect

Git can see the .env file. If someone shared this project online right now, the fake API keys would be visible to everyone. This is what happens when developers forget to protect their secrets.

Step 3 — Create .gitignore to hide it

YOUR PROMPT

Create a .gitignore file that ignores .env and .tmp/. Then show me git status again.

- ☐ .env is no longer visible in git status
☐ I understand what .gitignore protects
-

Exercise 3 — .env and .env.example

You will create a safe shareable template alongside the private file.

Step 1 — Create .env.example

The .example file shows the structure without any real values — safe to share with anyone.

YOUR PROMPT

Create a .env.example file with these blank fields: ANTHROPIC_API_KEY=NOTION_API_KEY=OPENAI_API_KEY=

- ☐ .env.example exists with blank fields

Step 2 — Compare the two files

YOUR PROMPT

Show me the contents of both .env and .env.example side by side.

Pause and reflect

.env has fake secrets in it. .env.example has the same structure but with nothing filled in. You share .env.example with your team. You never share .env with anyone.

Step 3 — Confirm .env is still hidden from Git

YOUR PROMPT

Show me git status. Which of these two files does Git track — .env or .env.example?

- ☐ Only .env.example is tracked by Git
- ☐ .env is hidden and protected

Exercise 4 — Git and Git commits

You will make two commits and see how Git builds a permanent history you can always return to.

Step 1 — Make your first commit

Save everything so far as a named snapshot.

YOUR PROMPT

Stage all tracked files and make a Git commit with the message: feat: initial structure with gitignore and env example

- ☐ First commit created

Step 2 — Add a real file and commit again

YOUR PROMPT

Create a file called notes.md with one line: These are my first notes. Then commit it with the message: feat: add notes file

- ☐ Second commit created

Step 3 — Read your history

YOUR PROMPT

Show me the full git log with commit messages and dates.

Pause and reflect

You have two commits. Two moments in time. Each one named, each one permanent. If you deleted every file right now, Git could restore everything to either of these snapshots. That is what Git is — a reliable memory for your project.

- ☐ I can see two commits in the log
- ☐ I understand what a commit is

Exercise 5 — CLAUDE.local.md

You will create a private instruction file that only lives on your machine.

Step 1 — Create CLAUDE.local.md

YOUR PROMPT

Create a file called CLAUDE.local.md with this content:
`# Local Overrides
My preferred response language is English.
Always keep answers under 3 paragraphs.`

- ☐ CLAUDE.local.md exists

Step 2 — Add it to .gitignore

This file should never leave your machine. Add it to .gitignore now.

YOUR PROMPT

Add `CLAUDE.local.md` to the `.gitignore` file. Then show me git status to confirm Git cannot see it.

- ☐ `CLAUDE.local.md` is hidden from Git

Step 3 — Understand what it does

YOUR PROMPT

What is the difference between `CLAUDE.md` and `CLAUDE.local.md`? Why would I keep one private?

Pause and reflect

`CLAUDE.md` is shared with the whole project and everyone on the team. `CLAUDE.local.md` is yours alone — your personal preferences that only apply on your machine.

Exercise 6 — `.claude/settings.local.json`

You will create the local settings file that Claude uses for machine-specific configuration.

Step 1 — Create the file

YOUR PROMPT

Create a folder called `.claude` and inside it create a file called `settings.local.json` with this content: `{}`

- ☐ `.claude/settings.local.json` exists

Step 2 — Add it to `.gitignore`

YOUR PROMPT

Add `.claude/settings.local.json` to `.gitignore`. Then confirm with git status.

- ☐ The file is protected from Git

Step 3 — See the full picture

YOUR PROMPT

Show me everything that is currently listed in `.gitignore`.

Pause and reflect

Look at your `.gitignore` list. Every item there is a file Git will never touch. You have built real protection around your project — secrets, private settings, local preferences, all invisible to the outside world.

Exercise 7 — `node_modules/`

You will simulate what `node_modules/` looks like and understand why it is never saved to Git.

Step 1 — Create a pretend `node_modules` folder

We will not install real packages — just create the folder to see what Git thinks of it.

YOUR PROMPT

Create a folder called `node_modules`. Inside it create three files: `package-a.js`, `package-b.js`, `package-c.js` — each with one line of text.

- ☐ `node_modules` folder exists with 3 files

Step 2 — Check Git status before protecting it

YOUR PROMPT

Show me `git status`. Does Git want to track the `node_modules` folder?

Pause and reflect

Git can see `node_modules` and wants to track it. In a real project this folder can contain 50,000+ files. Committing it would make your project huge and slow. It also gets auto-generated — anyone can recreate it with one command.

Step 3 — Add it to `.gitignore`

YOUR PROMPT

Add `node_modules/` to `.gitignore`. Then show `git status` to confirm it disappears.

- ☐ `node_modules/` is now invisible to Git
- ☐ I understand why it is never committed

Exercise 8 — .tmp/

You will use the .tmp/ folder as a scratch space and confirm nothing inside it ever gets saved to Git.

Step 1 — Create a scratch file in .tmp/

YOUR PROMPT

Create a folder called .tmp and inside it create a file called scratch.md with the text: rough notes — do not save

- ☐ .tmp/scratch.md exists

Step 2 — Add .tmp/ to .gitignore and verify

YOUR PROMPT

Add .tmp/ to .gitignore. Then show me git status — can Git see the scratch file?

- ☐ .tmp/ is invisible to Git

Step 3 — Final check — see everything Git ignores

YOUR PROMPT

Show me the complete .gitignore file. Then make a final commit with the message: feat: complete exercise structure

Pause and reflect

Your .gitignore is now protecting: .env (secrets), CLAUDE.local.md (private instructions), .claude/settings.local.json (local settings), node_modules/ (auto-generated packages), .tmp/ (scratch files). None of these will ever be accidentally shared.

Exercise complete

You have created real files and folders, initialised Git, built a .gitignore that protects secrets and private files, used .env and .env.example correctly, made proper Git commits, and understood what each term from the glossary actually does in practice.

Exercice Pratique Claude Code

Utilise Claude Code (Antigravity) pour compléter chaque tâche ci-dessous. Tape les prompts exactement comme indiqué, puis coche la case quand c'est fait. L'objectif est de sentir ce que chaque terme du glossaire fait vraiment — pas seulement de le lire.

Avant de commencer

Ouvre une session Claude Code. Tous les prompts vont directement dans Claude Code — pas dans le chat Claude.ai.

Exercice 1 — .gitkeep

Tu vas créer un dossier vide et découvrir pourquoi Git ne peut pas le voir — puis le corriger avec un fichier .gitkeep.

Étape 1 — Créer un dossier vide

Demande à Claude Code de créer un dossier appelé mon-archive sans rien à l'intérieur.

TON PROMPT

Crée un dossier appelé mon-archive. Ne mets aucun fichier à l'intérieur.

☐ Fait — le dossier existe sur mon ordinateur

Étape 2 — Demande à Git ce qu'il voit

Initialise un dépôt Git et demande à Claude Code ce que Git peut voir.

TON PROMPT

Initialise un dépôt Git dans ce dossier. Puis montre-moi ce que Git voit avec `git status`.

Pause - Réfléchis

Regarde le résultat. Git mentionne-t-il mon-archive ? Git ne peut pas voir un dossier vide — il est invisible. C'est exactement le problème que .gitkeep résout.

Étape 3 — Corriger avec .gitkeep

Ajoute un fichier .gitkeep pour rendre le dossier visible à Git.

TON PROMPT

Ajoute un fichier .gitkeep dans le dossier mon-archive. Puis montre-moi `git status` à nouveau.

- ☐ Git peut maintenant voir le dossier mon-archive
 - ☐ Je comprends pourquoi .gitkeep existe
-

Exercice 2 — .gitignore

Tu vas créer un fichier .gitignore et prouver qu'il cache complètement des fichiers à Git.

Étape 1 — Créer un fichier secret

Crée un fichier qui ne devrait jamais être partagé — un faux .env avec de faux identifiants.

TON PROMPT

Crée un fichier appelé .env avec ce contenu
:ANTHROPIC_API_KEY=fausse-cle-secrete-123NOTION_API_KEY=fausse-notion-456

- ☐ Le fichier .env existe

Étape 2 — Vérifier que Git peut le voir (danger !)

Avant de le protéger, confirme que Git suit actuellement ce fichier secret.

TON PROMPT

Montre-moi git status. Est-ce que Git peut voir le fichier .env en ce moment ?

Pause - Réfléchis

Git peut voir le fichier .env. Si quelqu'un partageait ce projet en ligne maintenant, les fausses clés API seraient visibles par tout le monde. C'est ce qui arrive quand les développeurs oublient de protéger leurs secrets.

Étape 3 — Créer .gitignore pour le cacher

TON PROMPT

Crée un fichier .gitignore qui ignore .env et .tmp/. Puis montre-moi git status à nouveau.

- ☐ .env n'est plus visible dans git status
 - ☐ Je comprends ce que .gitignore protège
-

Exercice 3 — .env et .env.example

Tu vas créer un modèle partageable et sécuriser à côté du fichier privé.

Étape 1 — Créer .env.example

Le fichier .example montre la structure sans aucune vraie valeur — sans danger à partager.

TON PROMPT

Crée un fichier .env.example avec ces champs vides
:ANTHROPIC_API_KEY=NOTION_API_KEY=OPENAI_API_KEY=

- ☐ .env.example existe avec des champs vides

Étape 2 — Comparer les deux fichiers

TON PROMPT

Montre-moi le contenu de .env et .env.example côte à côte.

Pause - Réfléchis

.env a de faux secrets. .env.example a la même structure mais sans rien de rempli. Tu partages .env.example avec ton équipe. Tu ne partages jamais .env avec qui que ce soit.

Étape 3 — Confirmer que .env est caché

TON PROMPT

Montre-moi git status. Lequel de ces deux fichiers Git suit — .env ou .env.example ?

- ☐ Seul .env.example est suivi par Git
- ☐ .env est caché et protégé

Exercice 4 — Git et les commits

Tu vas faire deux commits et voir comment Git construit un historique permanent.

Étape 1 — Faire ton premier commit

Sauvegarde tout ce qui existe jusqu'ici comme un instantané nommé.

TON PROMPT

Indexe tous les fichiers suivis et fais un commit Git avec le message : feat: structure initiale avec gitignore et env example

- ☐ Premier commit cree

Etape 2 — Ajouter un fichier et faire un second commit

TON PROMPT

Cree un fichier appele notes.md avec une ligne : Mes premieres notes. Puis committe-le avec le message : feat: ajout fichier notes

- ☐ Deuxieme commit cree

Etape 3 — Lire ton historique

TON PROMPT

Montre-moi le git log complet avec les messages de commit et les dates.

Pause - Reflexis

Tu as deux commits. Deux moments dans le temps. Chacun nomme, chacun permanent. Si tu supprimais tous les fichiers maintenant, Git pourrait tout restaurer. C'est ca Git — une memoire fiable pour ton projet.

- ☐ Je vois deux commits dans le log
- ☐ Je comprends ce qu'est un commit

Exercice 5 — CLAUDE.local.md

Tu vas creer un fichier d'instructions prive qui vit uniquement sur ta machine.

Etape 1 — Creer CLAUDE.local.md

TON PROMPT

Cree un fichier appele CLAUDE.local.md avec ce contenu :
Parametres locaux
Ma langue de reponse preferee est le francais.
Garde toujours les reponses sous 3 paragraphes.

- ☐ CLAUDE.local.md existe

Etape 2 — L'ajouter au .gitignore

Ce fichier ne doit jamais quitter ta machine. Ajoute-le au .gitignore maintenant.

TON PROMPT

Ajoute CLAUDE.local.md au fichier .gitignore. Puis montre git status pour confirmer que Git ne peut pas le voir.

☐ CLAUDE.local.md est cache de Git

Etape 3 — Comprendre la difference

TON PROMPT

Quelle est la difference entre CLAUDE.md et CLAUDE.local.md ? Pourquoi est-ce que je garderais l'un prive ?

Pause - Reflexis

CLAUDE.md est partage avec tout le projet et toute l'equipe. CLAUDE.local.md n'appartient qu'a toi — tes preferences personnelles qui s'appliquent uniquement sur ta machine.

Exercice 6 — .claude/settings.local.json

Tu vas creer le fichier de parametres locaux que Claude utilise pour la configuration specifique a ta machine.

Etape 1 — Creer le fichier

TON PROMPT

Cree un dossier appele .claude et a l'interieur cree un fichier appele settings.local.json avec ce contenu : {}

☐ .claude/settings.local.json existe

Etape 2 — L'ajouter au .gitignore

TON PROMPT

Ajoute .claude/settings.local.json au .gitignore. Puis confirme avec git status.

☐ Le fichier est protege de Git

Etape 3 — Voir la liste complete

TON PROMPT

Montre-moi tout ce qui est actuellement liste dans `.gitignore`.

Pause - Reflexis

Regarde ta liste `.gitignore`. Chaque element est un fichier que Git ne touchera jamais. Tu as construit une vraie protection autour de ton projet — secrets, parametres prives, preferences locales, tout invisible pour le monde exterieur.

Exercice 7 — node_modules/

Tu vas simuler ce a quoi ressemble `node_modules/` et comprendre pourquoi il n'est jamais sauvegarde dans Git.

Etape 1 — Creer un faux dossier node_modules

On n'installera pas de vrais paquets — juste creer le dossier pour voir ce que Git en pense.

TON PROMPT

Cree un dossier appele `node_modules`. A l'interieur cree trois fichiers : `paquet-a.js`, `paquet-b.js`, `paquet-c.js` — chacun avec une ligne de texte.

- ☐ Le dossier `node_modules` existe avec 3 fichiers

Etape 2 — Verifier git status avant de le proteger

TON PROMPT

Montre-moi `git status`. Est-ce que Git veut suivre le dossier `node_modules` ?

Pause - Reflexis

Git peut voir `node_modules` et veut le suivre. Dans un vrai projet, ce dossier peut contenir plus de 50 000 fichiers. Le commiter rendrait ton projet enorme et lent. Il est aussi genere automatiquement — n'importe qui peut le recreer avec une seule commande.

Etape 3 — L'ajouter au .gitignore

TON PROMPT

Ajoute `node_modules/` au `.gitignore`. Puis montre `git status` pour confirmer qu'il disparaît.

- ☐ node_modules/ est maintenant invisible pour Git
- ☐ Je comprends pourquoi il n'est jamais commite

Exercice 8 — .tmp/

Tu vas utiliser le dossier .tmp/ comme espace de brouillon et confirmer que rien a l'interieur n'est jamais sauvegarde dans Git.

Etape 1 — Creer un fichier de brouillon dans .tmp/

TON PROMPT

Cree un dossier appele .tmp et a l'interieur cree un fichier appele brouillon.md avec le texte : notes brouillon – ne pas sauvegarder

- ☐ .tmp/brouillon.md existe

Etape 2 — Ajouter .tmp/ au .gitignore et verifier

TON PROMPT

Ajoute .tmp/ au .gitignore. Puis montre git status – Git peut-il voir le fichier brouillon ?

- ☐ .tmp/ est invisible pour Git

Etape 3 — Verification finale

TON PROMPT

Montre-moi le fichier .gitignore complet. Puis fais un commit final avec le message : feat: structure exercice complete

Pause - Reflexis

Ton .gitignore protege maintenant : .env (secrets), CLAUDE.local.md (instructions privees), .claude/settings.local.json (parametres locaux), node_modules/ (paquets auto-generees), .tmp/ (fichiers brouillon). Aucun de ces fichiers ne sera jamais partage accidentellement.

Exercice termine

Tu as cree de vrais fichiers et dossiers, initialise Git, construit un .gitignore qui protege les secrets et les fichiers prives, utilise .env et .env.example correctement, fait de vrais commits Git, et compris ce que chaque terme du glossaire fait vraiment en pratique.